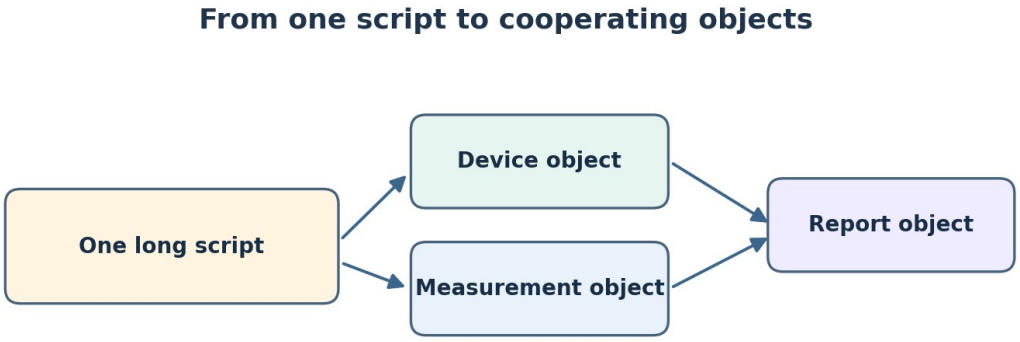
חוברת היסוד לספר הקורס המאוחד

פרק 2 — תכנות מונחה עצמים ב־Python

ממחלקה ועצם ועד למבנה של מערכת AI: הבנה, תכנון ומימוש

מסלול התמחות למידת מכונה ולמידה עמוקה | כיתות י״א–י״ב | מסגרת סילבוס: 30 שעות

מהדורת תלמיד • הסברים מדורגים • קוד עצמאי • המחשות מקוריות • תרגול והכנה לפרויקט



איור פתיחה: מעבר מסקריפט יחיד למספר עצמים בעלי תפקידים ברורים.

לפני שמתחילים — הסיפור שלנו

דודו אחראי על מעבדת המחשבים בבית הספר. עד היום הוא שמר את נתוני כל המחשבים ברשימות נפרדות: רשימת שמות, רשימת טמפרטורות ורשימת תקלות. כאשר נוספו מחשבים הוא התחיל להתבלבל: האם הטמפרטורה במקום השלישי שייכת למחשב השלישי בכל הרשימות? מה יקרה אם ימחק מחשב רק מרשימה אחת? כעת הוא רוצה לבנות תוכנה שאפשר להרחיב, לבדוק ולשלב בעתיד במערכת חיזוי תקלות מבוססת AI.

תכנות מונחה עצמים אינו אלגוריתם של למידת מכונה. זהו כלי לארגון תוכנה. הוא עוזר להפריד בין נתונים ופעולות, ליצור רכיבים שאפשר לבדוק בנפרד ולמנוע תלות מיותרת בין חלקי הפרויקט. בהמשך נשתמש באותן יכולות לכתיבת רכיבי קדם־עיבוד, מודלים ומערכות חיזוי.

מה תלמדו בפרק?

- להגדיר מחלקה, ליצור עצמים ולהבחין בין תכונה של עצם למשתנה רגיל.
- להסביר את התפקיד של self ושל __init__, ולכתוב פעולות שמבצעות חישוב אמיתי.
- להשתמש בהכלה ובהרכבה, ולהבחין ביניהן לבין ירושה ופולימורפיזם.
- להבין הפניה כפולה, העתקת עצמים, גישה מבוקרת לשדות וטיפול בקלט חריג.
- לבנות מערכת קטנה שבה רכיב נתונים ורכיב חיזוי פועלים יחד, ולבדוק את פעולתם.

תכנית הוראה מוצעת ל־30 שעות

תוצר תלמיד	נושאים ומיומנויות	מפגש (3 שעות)
מחלקת Device ראשונה	מהבעיה למחלקה, מצב והתנהגות	1
עצם עם חישוב ומצב	init__, self, get/set__ ופעולות	2
דוח על מספר מחשבים	פעולות פנימיות, רשימות עצמים	3
טבלת מעקב של עצמים	הפניות, העתקה וזהות	4
מערכת עצמים מקושרים	הכלה: Lab מכיל Device	5

תוצר תלמיד	נושאים ומיומנויות	מפגש (3 שעות)
	Measurement־ו	
מחלקה שמונעת מצב שגוי	כימוס, property ואימות נתונים	6
סוגי מחשבים ומימושים	ירושה, super ופולימורפיזם	7
קוד מאורגן וקובצי CSV	מודולים, קבצים וחריגות	8
Pipeline לימודי	רכיבי fit/transform/predict ובדיקות	9
SmartLab OOP	מיני־פרויקט, תיעוד והגנה בעל־פה	10

הקשר לשלוש תכניות הלימוד

דוגמה לשימוש	רמת הידע המכוונת	מסלול
קריאת נתוני מחשבים, סיכום והדפסתם	פונקציות, נתונים ומחברות Python בסיסיות	יחידה 3
עצם נתונים נושא וקטור מאפיינים	חישובים מתמטיים ואלגוריתמים (לא דרישת OOP עצמאית מזוהה)	נושאי ליבה — יחידה 5
Device, DataScaler, Predictor, Report	מחלקות, אחריות רכיבים, קוד מודלים ויישום תוכנה	התמחות למידה עמוקה

עוגן: במפת תכנית ההתמחות שבמאגר הקורס, פרק 2 — תכנות מונחה עצמים, 30 שעות. החלוקה למפגשים, דוגמת המעבדה וההקשרים ל־AI הם התאמה פדגוגית שלנו ולא חלוקה רשמית מאומתת של הסילבוס.

2.1 מדוע צריך תכנות מונחה עצמים?

נתאר שלושה מחשבים באמצעות שלוש רשימות: שמות, טמפרטורות ונתוני תקלות. כדי להפיק דוח על המחשב הראשון ניגש לאינדקס 0 בשלוש הרשימות. כל עוד הרשימות תמיד באותו אורך ובאותו סדר, התוכנית עובדת. אבל בפועל שינוי קטן — הוספה, מחיקה או מיון — עלול לשבור את ההתאמה ביניהן. הבעיה אינה שהרשימות הן כלי גרוע; הבעיה היא שאנחנו מייצגים ישות אחת באמצעות נתונים מפוזרים ללא הגדרה אחת שמאגדת אותם.

```
names = ["PC-01", "PC-02", "PC-03"]
temperatures = [58, 73, 64]
failures = [0, 2, 1]
for i in range(len(names)):
    print(names[i], temperatures[i], failures[i])
```

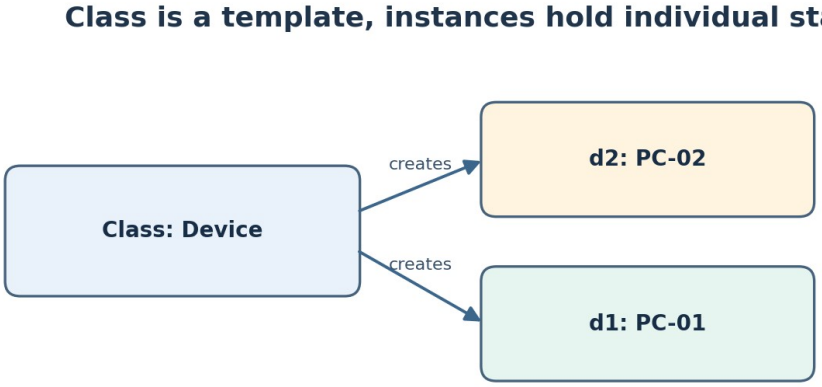
אפשר להחליף את שלוש הרשימות ברשימה של מילונים, וזה כבר שיפור. בתכנות מונחה עצמים עושים צעד נוסף: מגדירים מחלקה שמכילה את נתוני המחשב וגם פעולות שקשורות אליו. לדוגמה, במקום שקוד חיצוני יחליט בכל מקום אם הטמפרטורה חריגה, המחלקה עצמה יכולה להציע פעולה בשם needs_check. כך הכלל יוגדר פעם אחת ויוכל לשמש כמה חלקים בתוכנית.

☐ בדיקת הבנה: איזו בעיה עלולה להתרחש אם נוסיף שם של מחשב חדש אך נשכח להוסיף ערך לרשימת הטמפרטורות?

2.2 מחלקה, עצם, תכונות והתנהגות

מחלקה (Class) היא תיאור כללי של סוג עצמים; עצם (Object, או Instance) הוא מופע מסוים שנוצר על פי התיאור. אפשר לחשוב על מחלקה כתבנית להכנת כרטיסי מחשב: כל כרטיס מכיל אותם סוגי פרטים,

אבל הערכים בכל כרטיס שונים. התכונות (Attributes) מתארות את המצב של העצם; המתודות (Methods) מתארות מה הוא יודע לעשות.



איור 2.1: מחלקה אחת משמשת ליצירת כמה עצמים, ולכל עצם נתונים משלו.

```
class Device:
    def __init__(self, name, temperature):
        self.name = name
        self.temperature = temperature

    def needs_check(self):
        return self.temperature >= 70

computer_a = Device("PC-01", 58)
computer_b = Device("PC-02", 73)
print(computer_a.name, computer_a.needs_check())
print(computer_b.name, computer_b.needs_check())
```

כאשר מופיעה ההוראה Device("PC-01", 58), פייתון יוצרת מופע חדש ומפעילה את __init__ שלו. המופע שמתקבל נשמר במשתנה computer_a. שתי ההפניות computer_a ו־computer_b מצביעות על שני עצמים שונים, ולכן שינוי הטמפרטורה של אחד מהם אינו משנה את האחר.

דוגמה מהמעבדה	משמעות	רכיב בקוד
תבנית של כרטיס מחשב	הגדרה כללית של סוג העצם	class Device
מחשב מסוים	משתנה המחזיק הפניה לעצם	computer_a
שם המחשב	תכונה השייכת למופע	self.name
בדיקת סף טמפרטורה	פעולה של המופע	needs_check()

חשוב — הפעולה needs_check בדוגמה היא כלל תכנותי קבוע, ולא מודל שלמד מנתונים. בפרקים מתקדמים נחליף את הכלל הפשוט במודל שנאמן על נתוני אמת.

2.3 מה עושה __init__ ומה פירוש self?

הביטוי __init__ נקרא לעיתים "בנאי" בשפת ההוראה. ליתר דיוק, זוהי מתודת אתחול: היא מופעלת לאחר יצירת העצם ומגדירה את מצבו הראשוני. הפרמטר self מציין את המופע הנוכחי. כאשר קוראים computer_a.needs_check(), פייתון מעבירה את המופע בתור הפרמטר הראשון של המתודה — גם אם איננו כותבים אותו במפורש בעת הקריאה.

השוו בין name בלי self לבין self.name. השם name הוא פרמטר מקומי שקיים בזמן הפעלת __init__. לעומתו self.name נשמר כתכונה של המופע, ולכן אפשר לגשת אליו גם בהמשך, לאחר שהפעולה הסתיימה. ההבדל הוא כמו ההבדל בין פתק זמני לבין נתון ששמור בכרטיס של אותו מחשב.

```
class Device:
    def __init__(self, name, temperature=25):
        self.name = name
        self.temperature = temperature

    def description(self):
        return f"{self.name}: {self.temperature} C"

first = Device("PC-01")
second = Device("PC-02", 65)
print(first.description())
print(second.description())
```

בדוגמה הוגדר ערך ברירת מחדל temperature=25. לכן אפשר ליצור מחשב בלי למסור טמפרטורה, אך בכל זאת חייבים למסור שם. ערך ברירת מחדל צריך להתאים למשמעות הנתון; במערכת אמיתית עדיף לעיתים לציין שהמדידה אינה ידועה מאשר להמציא מדידה של 25 מעלות.

טבלת מעקב — יצירת שני עצמים

שלב	הוראה	התוצאה במצב העצמים
1	first = Device("PC-01")	first.name=PC-01, first.temperature=25
2	second = Device("PC-02",65)	second.name=PC-02, second.temperature=65
3	second.temperature = 72	רק second.temperature משתנה ל-72
4	first.description()	מחזירה את תיאור העצם הראשון

□ שאלת העברה: מה יקרה אם בתוך description נכתוב temperature במקום self.temperature? הסבירו לפי תחום ההכרה של המשתנים.

2.4 פעולות פנימיות: חישוב שמשתמש במצב העצם

מתודה אינה חייבת רק להדפיס נתון. היא יכולה לחשב תוצאה על סמך המצב של העצם או לעדכן אותו. נרצה למשל להוסיף קריאה חדשה של הטמפרטורה ולחשב את ההפרש מן המדידה הקודמת. כדי לעשות זאת נגדיר פעולה עם פרמטר חדש ונבחין בין ערך שהפעולה מחזירה לבין שינוי שהיא מבצעת במופע.

```
class Device:
    def __init__(self, name, temperature):
        self.name = name
        self.temperature = temperature

    def update_temperature(self, new_value):
        change = new_value - self.temperature
        self.temperature = new_value
        return change

pc = Device("PC-01", 60)
print(pc.update_temperature(68))
print(pc.temperature)
print(pc.update_temperature(65))
```

בקריאה הראשונה change הוא 8, והטמפרטורה נשמרת כ־68. בקריאה השנייה change הוא 3 −, כי הערך החדש קטן מן הקודם. חשוב לשים לב לסדר הפעולות: מחשבים את ההפרש לפני עדכון התכונה. אם נעדכן תחילה, החישוב ייתן תמיד אפס.

כיצד מתכננים מתודה?

לפני כתיבת כל מתודה מומלץ לענות על ארבע שאלות: מה היא מקבלת? מה היא משנה? מה היא מחזירה? ומה יקרה בקלט לא תקין? תכנון מראש מונע מתודות שמבצעות יותר מדי תפקידים ומקשה פחות על בדיקות היחידה.

שאלה	עבור **update_temperature**
קלט	טמפרטורה חדשה — new_value
מצב לפני	self.temperature
שינוי	עדכון self.temperature
פלט	ההפרש מן המדידה הקודמת

2.5 מספר עצמים — רשימת מחשבים ולולאות

מעבדה כוללת יותר ממחשב אחד. לכן ניצור רשימה של עצמי Device, נעבור עליהם בלולאה ונבנה דוח. המושג המרכזי הוא שהרשימה מחזיקה הפניות לעצמים; כל עצם אחראי לחשב האם הוא דורש בדיקה, ואילו הדוח אחראי לאסוף את התוצאות של כולם.

```
class Device:
    def __init__(self, name, temperature):
        self.name = name
        self.temperature = temperature

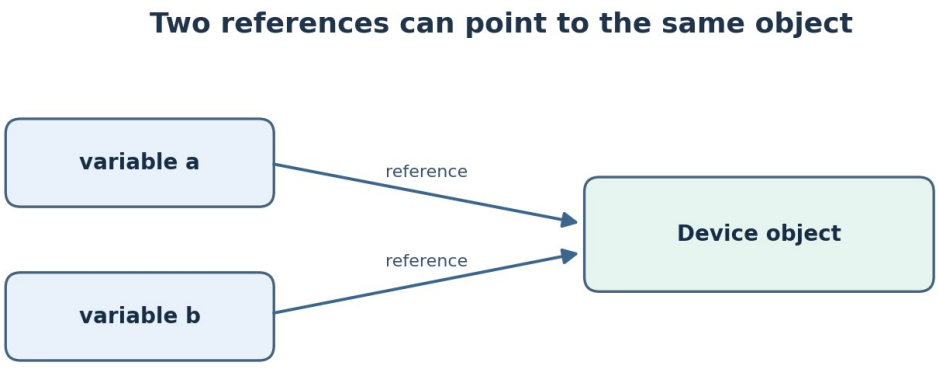
    def needs_check(self):
        return self.temperature >= 70

computers = [Device("PC-01", 61), Device("PC-02", 77),
             Device("PC-03", 72)]
for computer in computers:
    if computer.needs_check():
        print("CHECK:", computer.name)
```

בדוגמה הפעולה needs_check שייכת למחלקה Device, והלולאה רק מפעילה אותה. בעתיד נוכל להחליף את הכלל הפנימי בלי לשנות את לולאת הדוח. זו דוגמה ליתרון חשוב של OOP: מקומיות של שינוי.

2.6 זהות, הפניה כפולה והעתקה של עצמים

כאשר מבצעים b = a לא נוצרת אוטומטית העתקה של העצם. מתקבלת הפניה נוספת לאותו עצם, ולכן שינוי באמצעות b ייראה גם דרך a. זה שונה מעבודה עם מספרים פשוטים, שבהם המשתנה מקבל ערך, ולא עצם שניתן לשנות את שדותיו. כדי לבנות מערכות אמינות חשוב להבחין בין אותו עצם לבין עצם אחר בעל נתונים זהים.



איור 2.2: שני משתנים שונים מצביעים לאותו עצם בזיכרון.

```
class Device:
    def __init__(self, name):
        self.name = name

a = Device("PC-01")
b = a
b.name = "PC-99"
print(a.name)
print(a is b)

c = Device(a.name)
print(a is c)
print(a.name == c.name)
```

התוצאה a.name היא PC-99, מפני ששינוי דרך b שינה את אותו המופע. הביטוי a is b מחזיר True, כי זו אותה זהות. לעומת זאת c הוא מופע חדש, ולכן a is c מחזיר False גם אם לשניהם אותו שם. השוואת התכונות שלהם, לעומת זאת, יכולה להחזיר True. זהות אינה זהה לשוויון ערכים.

העתקה שטחית לעומת העתקה עמוקה

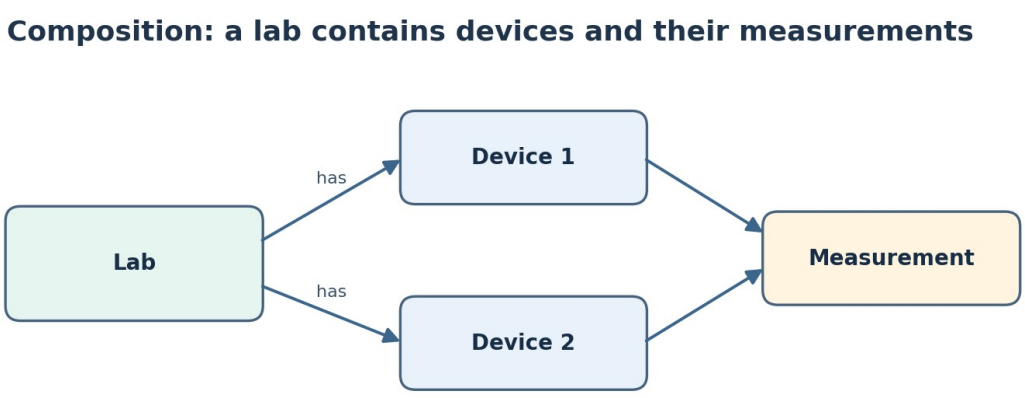
כאשר לעצם יש שדות פשוטים כמו מספר ומחרוזת, אפשר לעיתים ליצור עצם חדש באמצעות בנאי שמקבל את אותן תכונות. כאשר עצם מכיל רשימות או עצמים נוספים, העתקה מורכבת יותר: copy.copy מעתיק את האובייקט החיצוני, אבל בדרך כלל משאיר הפניות משותפות לעצמים פנימיים; copy.deepcopy מנסה להעתיק גם את הרכיבים המקוננים. אין צורך לשנן ספרייה זו, אך חשוב לדעת לשאול אם שני עצמים חולקים נתונים משותפים.

```
import copy
records = {"device": "PC-01", "temps": [60, 65]}
shallow = copy.copy(records)
deep = copy.deepcopy(records)
shallow["temps"].append(72)
print(records["temps"])
print(deep["temps"])
```

חשוב — עצם שנראה כעותק במסך אינו בהכרח בלתי תלוי. בפרויקט AI בעיות כאלה מופיעות בעת שינוי רשימות דוגמאות, הגדרות ניסוי או אובייקטים המשותפים למספר רכיבים.

2.7 הכלה: עצם שמכיל עצמים אחרים

בעולם האמיתי מחשב נמדד כמה פעמים, ומעבדה כוללת כמה מחשבים. לכן נכון ליצור מחלקת Measurement שמתארת מדידה, מחלקת Device שמחזיקה מדידות, ומחלקת Lab שמחזיקה מחשבים. זהו קשר של ״יש לו״ (has-a). הכלה מאפשרת לפרק מערכת ליחידות בעלות אחריות מוגדרת: מדידה שומרת תוצאה, מחשב אוסף מדידות, ומעבדה מפיקה דוח כללי.



איור 2.3: קשרי הכלה במערכת לניטור מעבדת מחשבים.

```
class Measurement:
    def __init__(self, value, minute):
        self.value = value
        self.minute = minute

class Device:
    def __init__(self, name):
        self.name = name
        self.measurements = []

    def add_measurement(self, value, minute):
        self.measurements.append(Measurement(value, minute))

    def average_temperature(self):
        if len(self.measurements) == 0:
            return None
        total = sum(m.value for m in self.measurements)
        return total / len(self.measurements)

a = Device("PC-01")
a.add_measurement(60, 0)
a.add_measurement(70, 10)
print(a.average_temperature())
```

נשים לב מדוע הרשימה נוצרת בתוך __init__ באמצעות self.measurements = []. כך כל מופע מקבל רשימה חדשה משלו. אם נגדיר רשימה משתנה ברמת המחלקה, או נשתמש ברשימה משתנה כערך ברירת מחדל, עלול להיווצר שיתוף לא רצוי בין עצמים. זו טעות נפוצה שכדאי לזהות כבר בשלב הלמידה.

הכלה לעומת ירושה

ביטוי עזר: Lab has Devices — המעבדה מכילה מחשבים, ולכן זה קשר הכלה. לעומת זאת Laptop is a Device — מחשב נייד הוא סוג של מחשב, ולכן ייתכן קשר ירושה. לפני כל קשר בין מחלקות שאלו: האם זהו רכיב שמצוי בתוך אחר, או סוג מיוחד של אותו דבר?

☐ שאלת חשיבה: האם רשת עצבית היא סוג של Dataset, או שהיא משתמשת ב־Dataset? נמקו לפי קשר is-a לעומת has-a.

2.8 כימוס, אימות נתונים ו־property

אם כל חלק בתוכנית יוכל להציב ערך שרירותי לטמפרטורה, אפשר לקבל נתונים לא הגיוניים. לכן נרצה שמחלקת Device תגדיר מקום אחד שבו בודקים קלט חדש. בפייתון נהוג לסמן שדות פנימיים בקו תחתון בתחילתם, למשל _temperature. זהו מוסכם בין מתכנתים ולא מנגנון אבטחה מוחלט; שמות עם קו תחתון הם עדיין נגישים מבחינה טכנית. המטרה היא שהקוד ישתמש בממשק ברור במקום לשנות את ייצוג הנתונים ישירות.

```
class Device:
    def __init__(self, name, temperature):
        self.name = name
        self.temperature = temperature

    @property
    def temperature(self):
        return self._temperature

    @temperature.setter
    def temperature(self, value):
        if not -30 <= value <= 120:
            raise ValueError("Temperature outside allowed range")
        self._temperature = value

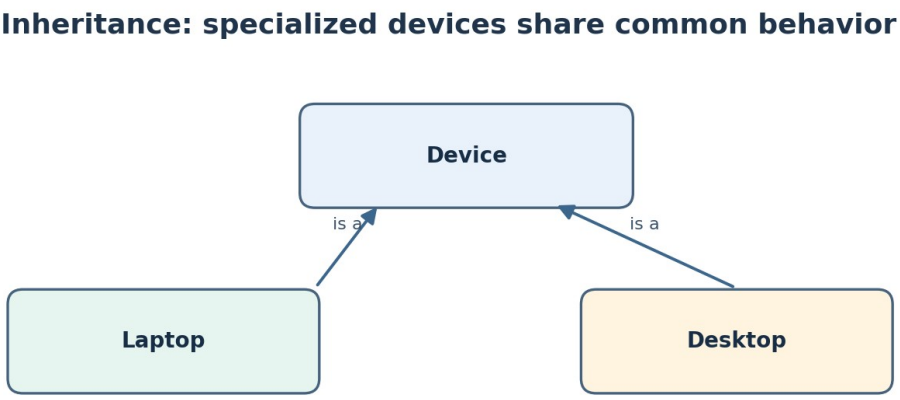
pc = Device("PC-01", 65)
pc.temperature = 71
print(pc.temperature)
```

הקוד בודק שהטמפרטורה בטווח שנבחר לדוגמה. מדובר בכלל אימות לצורכי הוראה, לא בטווח תקין רפואי או תקן של יצרן מחשבים. אם ננסה להציב 999 תיזרק חריגה מסוג ValueError. עדיף שהנתונים יידחו בשלב הקליטה מאשר שהתוכנית תמשיך לבצע חיזויים על ערכים שאין להם משמעות.

דגש — @property מאפשרת לקרוא תכונה כאילו הייתה שדה רגיל, אבל לבצע מאחורי הקלעים פעולה. זו אפשרות שימושית, לא חובה לכל שדה; מחלקה פשוטה אינה צריכה להפוך כל נתון ל־property ללא סיבה.

2.9 ירושה, super ופולימורפיזם

ירושה מאפשרת ליצור מחלקה מיוחדת שמבוססת על מחלקה כללית. אם לכל המחשבים יש שם וטמפרטורה, אבל למחשב נייד יש גם אחוז סוללה, אפשר לתאר Laptop כ־Device עם מידע נוסף. המחלקה היורשת מקבלת את המתודות והמאפיינים של מחלקת הבסיס, ויכולה להוסיף או לשנות התנהגות. חשוב לא להשתמש בירושה רק כדי לקצר כמה שורות קוד: קשר הירושה צריך להיות הגיוני מבחינת משמעות העצמים.



איור 2.4: *Laptop* ו־*Desktop* הם שני סוגים של *Device*.

```
class Device:
    def __init__(self, name):
        self.name = name

    def status(self):
        return "Active"

class Laptop(Device):
    def __init__(self, name, battery):
        super().__init__(name)
        self.battery = battery

    def status(self):
        if self.battery < 20:
            return "Charge needed"
        return super().status()

computers = [Device("PC-01"), Laptop("LAP-02", 12)]
for computer in computers:
    print(computer.name, computer.status())
```

הקריאה super().__init__(name) מפעילה את מתודת האתחול של מחלקת הבסיס. כך נשמרת הגדרת השם במקום אחד. ב־Laptop דרסנו (override) את המתודה status: בקריאה על מחשב נייד בעל סוללה חלשה יוחזר מסר אחר; בקריאה על Device רגיל יוחזר Active. זו דוגמה לפולימורפיזם: אותו שם מתודה מוביל להתנהגות מתאימה למופע המסוים.

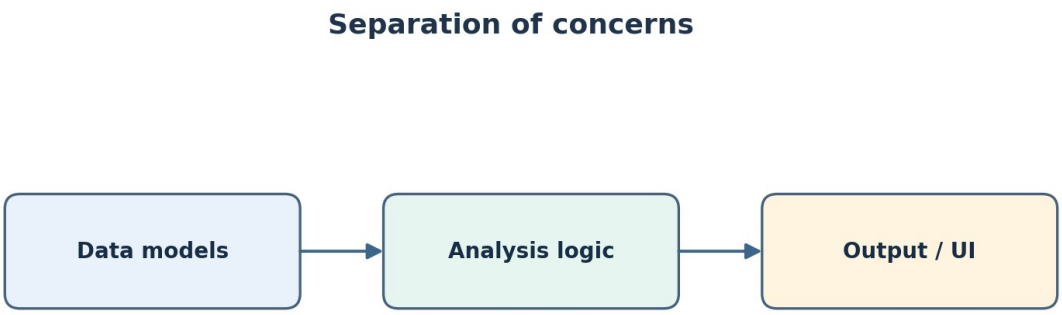
פולימורפיזם ללא קשר ירושה ישיר

פייתון מאפשרת גם כתיבת קוד שעובד עם כל עצם שמציע מתודה מתאימה, ללא חובה ששניהם יירשו ממחלקת בסיס. דרך חשיבה זו מכונה לעיתים duck typing. בספר שלנו נשתמש תחילה בירושה כי היא קלה להמחשה, ובהמשך נכיר ממשק של רכיבי AI שממממשים fit או predict בדרכים שונות.

☐ שאלת הבנה: מדוע שינוי של המתודה status במחלקת Laptop אינו מחייב שינוי בלולאה שעוברת על computers?

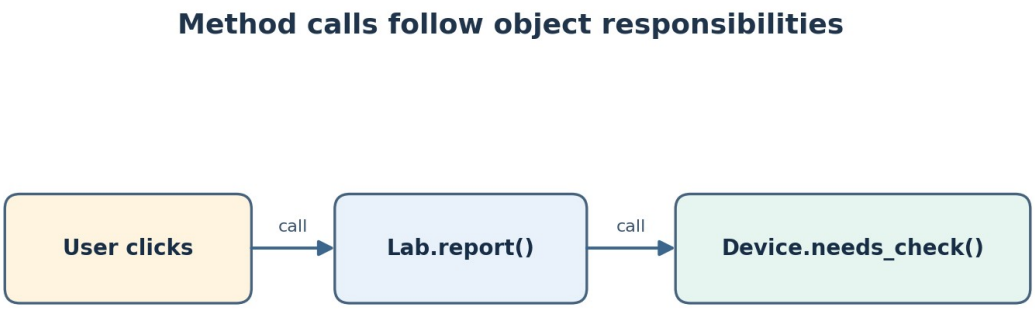
2.10 תרשים מחלקות ותכנון אחריות

לפני שמתחילים לכתוב פרויקט גדול, כדאי לשרטט אילו מחלקות דרושות, מי מחזיקה נתונים, מי מבצעת חישוב ומי מציגה תוצאה. כלל פשוט הוא "אחריות אחת ברורה לכל מחלקה". אין הכוונה שלמחלקה מותרת רק מתודה אחת; הכוונה שאפשר לתאר את מטרתה במשפט ברור. אם מחלקת Device גם קוראת CSV, גם מאמנת רשת וגם מציירת גרף, היא מבצעת תפקידים שונים מאוד וקשה לבדוק אותה.



איור 2.5: מומלץ להפריד בין מודלים של נתונים, לוגיקת ניתוח וממשק משתמש.

מחלקה	אחריות	פעולה אפשרית
Device	שמירת נתוני מחשב	add_measurement
Lab	ניהול אוסף מחשבים	add_device
DataScaler	לימוד טווחי מאפיינים והמרתם	fit / transform
Predictor	הפקת תחזית מדוגמה	predict
Report	הצגת תוצאות ומסקנות	build_summary



איור 2.6: כל מתודה מסייעת לאחרת בלי לבצע את כל העבודה בעצמה.

2.11 מודולים, קבצים ו־import

במחברת אחת אפשר להחזיק את כל הקוד, אבל בפרויקט אמיתי זו בדרך כלל בחירה לא נוחה. מודול בפייתון הוא קובץ py. המכיל הגדרות ומשפטי קוד. חלוקת התוכנית לקבצים מאפשרת לתקן רכיב בלי לחפש אותו בתוך מאות שורות ולבצע שימוש חוזר במחלקות שונות. בסביבת Colab אפשר להתחיל בתאים נפרדים, ובהמשך לעבור לקובצי .py.

שם קובץ	מה יכיל
device.py	המחלקות Device ו־Measurement
lab.py	המחלקה Lab והפעולות לניהול מחשבים
analysis.py	פעולות חישוב וסיכום נתונים
main.py	יצירת עצמים, הפעלה והצגת תוצאות

```
# file: device.py
class Device:
    def __init__(self, name):
        self.name = name

# file: main.py (save as a separate file)
# from device import Device
# computer = Device("PC-01")
# print(computer.name)
```

הדוגמה מציגה שני קבצים לצורכי תכנון, ולכן קטע main.py מובא כהערות כדי שניתן יהיה להריץ את התא בפני עצמו במחברת. כאשר עובדים בקבצים אמיתיים יש להסיר את סימני ההערה ולהריץ את main.py בסביבה שמכילה גם device.py. חשוב לא ליצור קובץ אישי בשם זהה לספרייה סטנדרטית כמו csv.py או math.py, כדי להימנע מהתנגשות בייבוא.

2.12 טיפול בטעויות — לא מתעלמים מקלט בעייתי

קוד אמין מבדיל בין מצב שאפשר לטפל בו לבין שגיאת תכנות שיש לתקן. לדוגמה, משתמש עשוי להקליד מחרוזת במקום טמפרטורה; אפשר לבקש ממנו להזין שוב. לעומת זאת, אם קראנו למתודה שאינה קיימת, אין טעם להסתיר את התקלה ולהמשיך כאילו דבר לא קרה. המנגנון try/except נועד לטפל במצבים צפויים ומוגדרים, ולא לכסות כל חריגה באופן גורף.

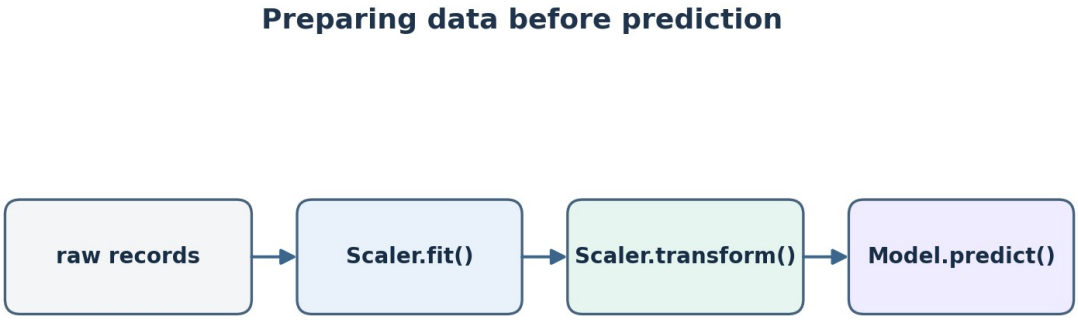
```
raw = "71"
try:
    temperature = float(raw)
    if not -30 <= temperature <= 120:
        print("Value outside the chosen range")
    else:
        print("Accepted:", temperature)
except ValueError:
    print("Enter a numeric temperature")
```

במערכת למידת מכונה יש לבצע בדיקות גם על אורך וקטור המאפיינים, על סדר העמודות ועל ערכים חסרים. מודל שמקבל ארבע תכונות בסדר מסוים אינו יכול לקבל בשקט שלוש תכונות או סדר אחר. עקרון זה נבנה בפרקים הבאים על הכנת נתונים ובדיקת מודלים.

2.13 OOP במערכת AI: הפרדה בין fit, transform ו־predict

נרצה כעת לראות את הקשר הישיר בין תכנות מונחה עצמים למערכות AI. בשפת העבודה של scikit-learn מקובל שרכיבי עיבוד נתונים מציעים פעולת fit הלומדת נתונים הדרושים להמרה, ופעולת transform הממירה דוגמאות באמצעות הערכים שנלמדו. מסווג מציע fit לאימון ו־predict לחיזוי. כאן נכתוב מחלקת MinMaxScaler לימודית קטנה כדי להבין את הרעיון, אך בפרויקט אמיתי נעדיף רכיבים מקצועיים שנבדקו היטב.

- מהדורת תלמיד • Python ב־OOP



איור 2.7: הרכיבים השונים משתתפים בשלבי הכנת הנתונים והחיזוי.

```
class SimpleMinMax:
    def __init__(self):
        self.minimum = None
        self.maximum = None

    def fit(self, values):
        if len(values) == 0:
            raise ValueError("Empty training data")
        self.minimum = min(values)
        self.maximum = max(values)
        return self

    def transform(self, values):
        if self.minimum is None:
            raise ValueError("Call fit before transform")
        span = self.maximum - self.minimum
        if span == 0:
            return [0.0 for _ in values]
        return [(v-self.minimum)/span for v in values]

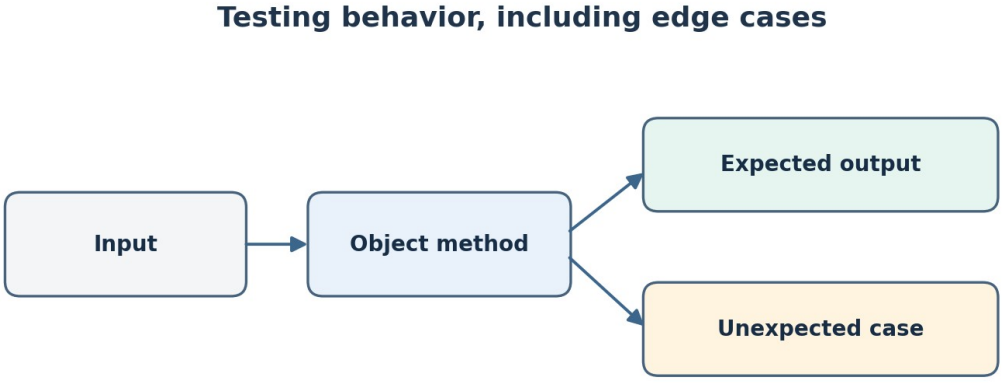
scaler = SimpleMinMax()
scaler.fit([40, 60, 80])
print(scaler.transform([50, 90]))
```

הפעולה fit לומדת מן האימון מינימום 40 ומקסימום 80. לאחר מכן transform ממירה 50 לערך 0.25 ואת 90 לערך 1.25. האחרון גדול מ־1 — וזה תקין, מפני שדוגמה חדשה יכולה להיות מחוץ לטווח האימון. אסור להתאים את המינימום והמקסימום מחדש על נתוני Test: פעולה כזו תחשוף למערכת מידע שלא אמור היה להיות זמין בזמן האימון.

הבחנה מרכזית — מחלקת SimpleMinMax היא הדגמה לימודית של עקרון התאמת טרנספורמציה. היא אינה מחליפה את בדיקות הקלט, הטיפול בחסרים וההתנהגות המלאה של MinMaxScaler המקצועית.

2.14 בדיקות יחידה ואיכות קוד

מחלקה טובה אינה רק מחלקה שנראית מסודרת. צריך להראות שהמתודות שלה פועלות נכון על קלט רגיל וגם במקרי קצה. בדיקת יחידה מפעילה רכיב קטן ומוודאת שהתוצאה מתאימה למה שציפינו. כאשר מתכננים בדיקות מראש, ההסבר של המתודות נעשה מדויק יותר: התלמיד חייב לחשוב מה פירוש ״נכון״ לפני שהוא רואה את הפלט.



איור 2.8: בדיקה משווה תוצאה בפועל לתוצאה צפויה, כולל מצבים חריגים.

```
scaler = SimpleMinMax()
scaler.fit([10, 20, 30])
result = scaler.transform([10, 15, 30])
assert result == [0.0, 0.25, 1.0]
assert scaler.transform([40]) == [1.5]
print("Tests passed")
```

הפקודה assert בודקת תנאי; אם הוא אינו מתקיים, התוכנית תעלה AssertionError. בדיקות מסוג זה שימושיות ללמידה ולבדיקות קטנות. במערכת גדולה אפשר להשתמש גם במסגרת בדיקות כמו pytest. חובה לא להסתפק במבחן אחד: יש לבדוק רשימת אימון ריקה, מאפיין קבוע, ערכי קצה, וכמות מאפיינים לא צפויה.

בדיקה חזותית של שינוי מצב

בנוסף לבדיקות חישוביות, חשוב לוודא שהממשק הציבורי של העצם לא מאפשר מצב סותר. לדוגמה, לא אמור להחזיק ממוצע טמפרטורה שנשמר במשתנה אחד בזמן שרשימת המדידות מתעדכנת במקום אחר. לעיתים עדיף לחשב את הממוצע מתוך הרשימה בכל קריאה, ולא לשמור עותק נוסף שדורש סנכרון. זו דוגמה לחשיבה הנדסית המתאימה במיוחד לתחום הנתונים.

2.15 מיני־פרויקט מסכם: SmartLab OOP

במיני־פרויקט דודו מבקש לבנות מערכת לניהול מדידות ממחשבי המעבדה. המערכת אינה צריכה לאמן מודל עמוק — זה יגיע בהמשך — אבל המבנה שלה צריך לאפשר להחליף בעתיד את כלל הסף הפשוט במסווג למידת מכונה. לכן נפריד בין אחסון המדידות, ניהול המעבדה, והפקת החלטת החיזוי.

רכיב	דרישה בסיסית	הרחבה אפשרית
Measurement	שמור ערך וזמן מדידה	תקינות טווח
Device	שמור שם ורשימת מדידות	חישוב מינימום, מקסימום וממוצע
Lab	הוסף מחשבים; הצג דוח	חיפוש לפי מזהה
ThresholdPredictor	החזר 0/1 לפי סף מוגדר	סף נפרד לסוג מחשב
Report	הצג דוח קריא	ייצוא CSV

```
class ThresholdPredictor:
    def __init__(self, threshold):
        self.threshold = threshold

    def predict(self, value):
        return int(value >= self.threshold)

predictor = ThresholdPredictor(70)
print(predictor.predict(65))
print(predictor.predict(74))
```

החלפנו כלל if שנכתב ישירות בתוך הדוח בעצם Predictor בעל מתודה predict. זו עדיין אינה למידת מכונה: הסף נקבע בידי המתכנת ואינו נלמד מנתונים. אבל המבנה כבר דומה לממשק של מודל אמיתי, כך שבהמשך נוכל להשתמש במחלקה שמאמנת פרמטרים ובוחנת נתונים חדשים.

מבדק קבלה למיני־פרויקט

- ☐ המערכת מציגה דוח עבור שלושה מחשבים, שלכל אחד מספר שונה של מדידות.
- ☐ המערכת אינה קורסת כאשר אין מדידות; היא מחזירה ערך מוסכם ומוסבר במקום ממוצע מדומה.
- ☐ מחלקת הניבוי מקבלת קלט אחד ומחזירה החלטה עקבית, בלי להדפיס מידע שלא התבקשה להדפיס.
- ☐ התלמיד יכול להסביר באילו מקומות יש הכלה ובאילו מקומות, אם בכלל, יש צורך בירושה.
- ☐ כל רכיב נבדק לפחות פעם אחת באמצעות קלט רגיל ופעם אחת באמצעות מקרה קצה.

העמקה 2.15.1 — משתנה של מחלקה לעומת משתנה של מופע

עד כה לכל מחשב הייתה טמפרטורה משלו: self.temperature. אבל לפעמים צריך נתון משותף לכל המחלקה, למשל מספר המופעים שנוצרו. במקרה כזה אפשר להגדיר משתנה מחלקה בראש המחלקה, מחוץ לכל המתודות. ההבחנה חשובה מאוד: משתנה מופע מתאר עצם אחד; משתנה מחלקה שייך למחלקה עצמה, אף שניתן לראות אותו גם דרך מופעים מסוימים.

```
class Device:
    created_count = 0

    def __init__(self, name):
        self.name = name
        Device.created_count += 1

a = Device("PC-01")
b = Device("PC-02")
print(Device.created_count)
print(a.name, b.name)
```

הספירה היא 2 מפני שמתודת האתחול הופעלה פעמיים. לעומת זאת a.name ו־b.name שונים, משום שכל שם נשמר במופע המתאים. אם נגדיר measurements = [] כמשתנה מחלקה, כל המופעים עלולים לחלוק אותה רשימה. מבחינה לשונית שתי הגישות נראות דומות; מבחינת התנהגות הן שונות לגמרי.

מתי להשתמש?	משמעות	מקום ההגדרה
מדידה של מחשב מסוים	מצב נפרד לכל מופע	self.temperature בתוך __init__
ספירת מופעים לימודית	נתון משותף למחלקה	created_count בגוף המחלקה
היסטוריית מדידות נפרדת	רשימה חדשה לכל מופע	self.measurements = [] בתוך __init__
בדרך כלל לא מתאים להיסטוריה אישית	רשימה משותפת בין מופעים שלא הסתירו אותה	measurements = [] בגוף המחלקה

כדי לבחון את התוצאה, נסו לפני ההרצה לנבא: אם למחשב הראשון נוספה מדידה, האם היא אמורה להופיע במחשב השני? אם התשובה בפועל חיובית בלי שהתכוונו לכך, ככל הנראה נשמרה רשימה

משותפת במקום רשימת מופע. זהו באג חשוב במיוחד כאשר עובדים עם מערכי נתונים של כמה משתמשים או ניסויים.

העמקה 2.15.2 — מעקב ידני אחרי רצף הפעלת מתודות

היכולת החשובה ביותר במבחן על עצמים אינה לזכור מילים באנגלית, אלא לדעת לעקוב אחר מצב העצם. נציג מעקב מלא ללא מחשב: בתחילה דודו יוצר מחשב ללא מדידות, לאחר מכן מוסיף שתי מדידות, שומר הפניה נוספת ולבסוף מוסיף מדידה דרך ההפניה השנייה. נעקוב גם אחר הערך המוחזר וגם אחר השינוי בזיכרון.

```
class Device:
    def __init__(self, name):
        self.name = name
        self.values = []

    def add(self, value):
        self.values.append(value)

    def average(self):
        if not self.values:
            return None
        return sum(self.values) / len(self.values)

first = Device("PC-01")
first.add(60)
first.add(80)
second = first
second.add(70)
print(first.values)
print(first.average())
```

חשוב במיוחד: ההוראה second = first אינה יוצרת עותק חדש. לכן הוספת 70 דרך second משנה את הרשימה שמוחזקת באותו עצם שאליו מתייחס גם first. תוצאת הממוצע תהיה 70, אבל התלמיד צריך להראות כיצד הגיע אליה ולא להסתפק בכך שהמחשב הדפיס אותה.

שלב	אילו נתונים נשמרים בעצם?	מה מחזירה **average**?
מיד אחרי first = Device(…)	[]	None
לאחר first.add(60)	[60]	60
לאחר first.add(80)	[60,80]	70
לאחר second = first	[60,80]	70
לאחר second.add(70)	[60,80,70]	70

שימו לב גם למקרה שבו אין מדידות. בתוכנית שלנו מחזירים None ולא אפס. החזרת אפס תיצור רושם שממוצע הטמפרטורות הוא 0 מעלות, בעוד שבפועל אין נתונים כלל. בחירת ערך החזרה מתאים היא חלק מתכנון ממשק המחלקה, ולא רק שאלה של תחביר.

העמקה 2.15.3 — תכנון לפני קוד: איך מחלקים מערכת לאחריות?

תלמידים רבים מתחילים לכתוב מחלקות לפי שמות העצמים שהם רואים בסיפור, אבל שוכחים לתכנן אילו פעולות כל מחלקה אמורה לבצע. נתרגל תהליך של ארבעה צעדים: ראשית מזהים את הישויות המרכזיות; שנית מנסחים את אחריותה של כל ישות; שלישית בוחרים איזה מידע צריך להיות שמור בה; ולבסוף מגדירים מתודות שיאפשרו לרכיבים אחרים להשתמש בה בלי לדעת את כל פרטי המימוש.

צעד	שאלת תכנון	החלטה בדוגמה
1 — זיהוי	מהן הישויות שקיימות בסיפור?	מחשב, מדידה, מעבדה, מסווג
2 — אחריות	מה כל אחד צריך לדעת לעשות?	Device שומר מדידות; Lab מנהלת מחשבים

צעד	שאלת תכנון	החלטה בדוגמה
3 — מידע	מהו מצב העצם ומה מחושב מחדש?	רשימת מדידות נשמרת; ממוצע מחושב
4 — ממשק	אילו פעולות חלקים אחרים יפעילו?	add, average, predict, summary

נבחן שתי הצעות לתכנון. בהצעה הראשונה Device מדפיס את כל הדוח ומאמן גם מודל. בהצעה השנייה Device שומר מדידות, Report מטפל בתצוגה ו־Predictor מחזיר תחזית. ההצעה השנייה נוחה יותר להרחבה, אך גם היא אינה ערובה לאיכות — חלוקה למחלקות קטנות מדי עלולה לפזר את הלוגיקה ולהקשות על ההבנה. המטרה היא חלוקה הגיונית לפי אחריות, לא מספר גדול ככל האפשר של מחלקות.

בפרויקט למידת מכונה נוכל בעתיד להחליף את Predictor במודל CNN או ברגרסיה מבלי לשכתב את המחלקה ששומרת את נתוני המחשב. כאשר ממשק predict ברור ונשאר יציב, שינוי המימוש הפנימי מצריך פחות שינויים במקומות אחרים. זהו שימוש מעשי חשוב בעקרון ההפרדה בין הממשק לבין אופן ביצוע העבודה.

העמקה 2.15.4 — כתיבת קוד קריא עם טיפוסים ותיעוד

ב־Python אפשר להוסיף רמזי טיפוסים (Type Hints) כדי לתאר מה מתודה מקבלת ומה היא מחזירה. לדוגמה, אפשר לציין שהמדידה היא מספר עשרוני והפעולה מחזירה אמת או שקר. רמזי הטיפוסים אינם מבצעים אכיפה מלאה בזמן ריצה; יש להמשיך לבדוק קלט בפועל כאשר זה חשוב. יתרונם העיקרי בספר שלנו הוא שהם הופכים את הממשק לקל יותר להבנה ומאפשרים לכלי פיתוח להתריע על חלק מהטעויות.

```
class ThresholdPredictor:
    def __init__(self, threshold: float):
        self.threshold = threshold

    def predict(self, value: float) -> bool:
        """Return True when the value reaches the threshold."""
        return value >= self.threshold

predictor = ThresholdPredictor(70.0)
print(predictor.predict(75.0))
```

השורה בתוך שלוש המרכאות היא Docstring — תיאור קצר של הפעולה. ההסבר אינו מתאר כיצד מחשבים באמצעות כל שורת קוד, אלא מה הפעולה מבטיחה למשתמש בה. כאשר תלמיד יפתח בעתיד מודל באמצעות PyTorch, תיעוד כזה יסייע לו להסביר את תפקיד רכיבי המערכת בפני בוחן ויקל על בדיקת הקוד.

העמקה 2.15.5 — תכנית עבודה למיני־פרויקט בארבע אבני דרך

כדי לאפשר למידה עצמאית ולא רק העתקת דוגמה, מומלץ לפתח את SmartLab בהדרגה. בכל אבן דרך נדרשת הוכחה שהרכיב הקודם עובד לפני שמוסיפים עוד חלק. כך קל יותר לגלות היכן נוצרה טעות, והתלמיד נדרש להסביר בכל שלב מה חדש ומה עדיין חסר.

אבן דרך	מה בונים?	בדיקת הצלחה
א׳ — נתונים	Measurement ו־Device עם מדידות	לכל עצם נשמרת היסטוריה עצמאית
ב׳ — ניהול	Lab שמכילה כמה מחשבים	אפשר להוסיף, לאתר ולספור מחשבים
ג׳ — חיזוי	ThresholdPredictor בעל predict	סיווג עקבי לפי סף מוגדר
ד׳ — הנדסה	Report, בדיקות ותיעוד	המערכת פועלת מחדש

בדיקת הצלחה	מה בונים?	אבן דרך
ומסבירה תוצאות		

הערכת הביצוע אינה נשענת רק על מספר הפקודות שהתלמיד כתב. נבדוק אם יש חלוקת אחריות נכונה, אם האובייקטים אינם משתפים מידע בטעות, אם קיימות בדיקות למצבי קצה ואם ניתן להסביר כיצד יעבור הקוד לשימוש במודל נלמד. חשוב להדגיש: הקוד המצורף לספר מדגים עקרונות; הגשת המיני־פרויקט דורשת מהתלמיד לכתוב פתרון משלו ולתעד החלטות.

2.16 טעויות נפוצות — כיצד מזהים ומתקנים?

דרך חשיבה לתיקון	למה זו בעיה?	הטעות
שאלו היכן המידע צריך להישמר	הפרמטר המקומי אינו בהכרח קיים במתודה אחרת	כתיבת name במקום self.name
צור רשימה חדשה בתוך __init__	מדידות של מחשב אחד מופיעות אצל אחר	שימוש ברשימה משותפת לכל העצמים
צרו מופע חדש או העתיקו במודע	שתי הפניות לאותו עצם	כדי להעתיק b = a
בדקו משמעות: הוא סוג של או מכיל?	המחלקות אינן מייצגות יחס is-a	ירושה במקום הכלה
הפרידו חישוב מהצגה	קשה לבדוק ולעשות שימוש חוזר	כל המחלקה מדפיסה וגם מחשבת
התאימו עיבוד על אימון בלבד	דליפת מידע בהכנת נתונים	fit על כל הנתונים לפני Test
טפלו רק בשגיאות צפויות	באגים מתגלים מאוחר מדי	except הסתרת כל חריגה עם כללי

2.17 תרגול מדורג — ללא פתרונות

רמה א׳ — זיהוי והבנת מושגים

☐ הסבירו במילים שלכם את ההבדל בין מחלקה, מופע, תכונה ומתודה. תנו דוגמה חדשה שאינה מחשב.

☐ עבור Device("PC-07", 68), כתבו מה מתרחש בעת יצירת המופע ואיזה מידע נשמר בו.

☐ בתוך __init__ מופיע גם name וגם self.name. מדוע צריך את שניהם?

☐ בתוכנית b = a ולאחר מכן "b.name = "X: מה יהיה a.name? נמקו בלי להריץ קוד.

רמה ב׳ — חישוב, מעקב ותכנון

☐ בנו מחלקת Sensor המקבלת שם וערך מדידה ומחזירה True כאשר המדידה חריגה לפי סף שנמסר.

☐ בנו טבלת מעקב לשלוש קריאות ל־update_temperature; בכל שלב ציינו מצב קודם, קלט, פלט ומצב חדש.

☐ צרו מחלקת Lab שמכילה לפחות שלושה Device, וכתבו פעולה המונה כמה מהם דורשים בדיקה.

☐ כתבו בדיקות עבור SimpleMinMax, כולל דוגמה שבה כל ערכי האימון זהים ודוגמה חדשה החורגת מטווח האימון.

רמה ג׳ — חשיבה של מפתח AI

☐ סרטטו תרשים מחלקות עבור מערכת שמזהה תקלות על סמך מדידות: אילו רכיבים הם נתונים ואילו מבצעים חיזוי?

☐ השוו שתי הצעות: מחלקה אחת של 250 שורות לעומת ארבע מחלקות בעלות אחריות ברורה. הציגו יתרונות ומגבלות לשתי החלופות.

☐ דונו מדוע לא נכון לקרוא לכללים מבוססי threshold בשם מודל למידת מכונה אם הסף לא נאמד מנתונים.

☐ הציעו שני מקרי בדיקה שיגלו באגים בהכנת נתונים לפני predict, והסבירו מה יקרה אם לא נבדוק אותם.

2.18 שאלת הערכה מסכמת בסגנון בחינה

מערכת SmartLab מכילה אוסף של עצמי Device. כל Device שומר שם, רשימת מדידות וטמפרטורה אחרונה. המחלקה Lab מפיקה דוח על כל המחשבים, והמחלקה ThresholdPredictor מסווגת את המחשבים לפי סף. לא כל קטע קוד המופיע בהמשך הוא מימוש של למידת מכונה; בחלק מן הסעיפים מדובר בסיווג לפי כלל קבוע.

סעיף א׳: הסבירו את הקשר בין Lab ל־Device ואת הקשר בין Device ל־Measurement. בחרו עבור כל קשר הכלה או ירושה ונמקו. סעיף ב׳: נתון שלמשתנים a ו־b יש אותה הפניה ל־Device. לאחר הוספת מדידה דרך b, כיצד ישתנה הממוצע המחושב דרך a? סעיף ג׳: תכננו מתודה שמחזירה את הממוצע בלי לשנות את מצב העצם. סעיף ד׳: הסבירו כיצד אפשר להחליף את ThresholdPredictor במודל שנלמד מנתונים תוך שמירה על ממשק predict. סעיף ה׳: הציעו שני ניסויי בדיקה ואיום אחד על אמינות המערכת.

בעת פתרון נדרשת דרך חשיבה ולא רק תשובה סופית. כתבו אילו הנחות קיבלתם, מתי מתבצע שינוי מצב ומה ניתן להסיק מן הפלט. השאלה מקורית בסגנון הערכה לימודית, ולא שאלה רשמית משאלון בגרות.

2.19 סיכום מושגים ודף נוסחאות

מושג	פירוש קצר
Class	תבנית המגדירה נתונים ומתודות
Instance/Object	מופע יחיד שנוצר ממחלקה
Attribute	נתון השייך לעצם
Method	פעולה של עצם
self	הפניה למופע שעליו פועלת המתודה
__init__	מתודת אתחול בעת יצירת מופע
Encapsulation	ארגון מידע וגישה מבוקרת אליו
Composition	עצם המכיל עצמים אחרים
Inheritance	מחלקה מיוחדת היורשת התנהגות ממחלקת בסיס
Polymorphism	אותו ממשק מפעיל התנהגויות שונות
Identity	האם שתי הפניות מצביעות לאותו עצם
fit/transform	למידת פרמטרי הכנה והמרת נתונים
predict	הפקת תחזית ממידע קלט
Unit test	בדיקה של רכיב תוכנה קטן

אין נוסחה מתמטית יחידה ל־OOP: ההצלחה נמדדת ביכולת לתכנן אחריות, להסביר מצב של עצמים, לבצע בדיקות ולהשתמש בקוד באופן אמין. כאשר נרצה לבצע חישוב מספרי, נוכל ליישם אותו בתוך מתודה תוך שמירה על ההפרדה בין נתונים ללוגיקה.

2.20 מקורות והמשך הדרך

עוגן בתכנית: מפת התמחות למידת מכונה ולמידה עמוקה — פרק 2, תכנות מונחה עצמים, 30 שעות: מסלול . https://github.com/samiabo75/Data-Science-ML-Course/tree/main/03-Deep-Learning-5-Units ההכנה המלווה: https://github.com/samiabo75/Data-Science-ML-Course/tree/main/01-Python-Basics/ 03-Deep-Learning-Python

מקורות מקצועיים: The Python Tutorial — Classes (docs.python.org/3/tutorial/classes.html); Python Data Model (docs.python.org/3/reference/datamodel.html); Python documentation — copy, dataclasses; Effective Python — Brett Slatkin, מהדורות מעודכנות; Hands-On Machine Learning with Scikit-Learn and PyTorch — Aurélien Géron, O’Reilly 2025. הספרים והתיעוד משמשים להעמקה מושגית ולהשראה פדגוגית; דוגמאות SmartLab, ההסברים והתרגילים נכתבו באופן מקורי.

הפרק הבא בחוברת היסוד: פרק 3 — מבוא למדעי הנתונים ולמידת מכונה (10 שעות). בו נחבר את מחלקות הנתונים והתכנות אל שאלת מחקר, מאגר נתונים, מודל, אימון וחיזוי.

דף סיום אישי — האם אני מוכן לפרק הבא?

לפני המעבר למבוא למדעי הנתונים, דרגו את שליטתכם בכל מיומנות. בחרו דוגמת קוד משלכם שמוכיחה לפחות שני סעיפים. אם אחת המיומנויות אינה ברורה, חזרו לדוגמה ולתרגול המתאימים; עדיף לזהות פער עכשיו מאשר באמצע פרויקט גדול.

מיומנות נבדקת	שולט/ת	דורש/ת תרגול
אני יוצר מחלקה ומסביר את תפקיד __init__ ו־self	☐	☐
אני מבדיל בין משתנה מופע למשתנה מחלקה	☐	☐
אני יודע להסביר הפניה כפולה והעתקת עצם	☐	☐
אני מזהה קשר הכלה לעומת קשר ירושה	☐	☐
אני מממש מתודה שמשנה מצב ומחזירה ערך	☐	☐
אני מתכנן מחלקה בעלת אחריות מוגדרת	☐	☐
אני בודק קוד בקלט רגיל ובמקרה קצה	☐	☐
אני מבדיל בין כלל קבוע לבין מודל שלמד מנתונים	☐	☐

משימת רפלקציה קצרה: כתבו שני משפטים על החלטת תכנון אחת שתחסוך לכם עבודה במיזם AI בעתיד. הוסיפו דוגמה אחת לבאג שזיהיתם בעצמכם, כיצד מצאתם אותו ומה שיניתם כדי לתקן. המטרה אינה לכתוב תשובה ״יפה״ אלא להראות למידה עצמאית מתוך התנסות.